Last One Awake — Project Plan & Team Split

Oct 8, 2026 · @Rushi

At a glance

The work splits into four workstreams with one owner each, divided along the code so nobody waits on anyone else. A complete, playable round exists by Oct 22, two weeks from now. The full game, proofs included, is done by Nov 5, ahead of Nov 6 (assumed mid-term date).

Kickoff is this weekend (W0, Oct 9–11). Weeks then run Monday to Sunday from W1 (Oct 12), and W4 is a short buffer week that ends with everything done on Nov 5.

What we cut to fit four weeks

Fitting nine weeks of planned work into four nearly doubles everyone's weekly hours, even after these cuts. The pillar work and all four proofs stay.

Cut	Instead	Frees
Could-haves: sound, difficulty, replay	Only if W4 has spare time	D
Floor-grid navigation	A* over 8–12 hand-placed waypoints around the furniture	A and C
Custom GLSL lamp and moonlight shaders	Three.js lights: one shadow-casting lamp, a moonlight fill, a faked light shaft	D
A full room of props	Four clusters: desk stack, bookshelf, nightstand with lamp, plant	B
Fracture for every breakable	Mugs and plates only, 2–4 pre-cut pieces each	A
A weekly sync	A written check-in every evening and merges to `main` every day	Everyone

If the course has checkpoints after Nov 5, those weeks go to the Could-haves and polish.

Who owns what

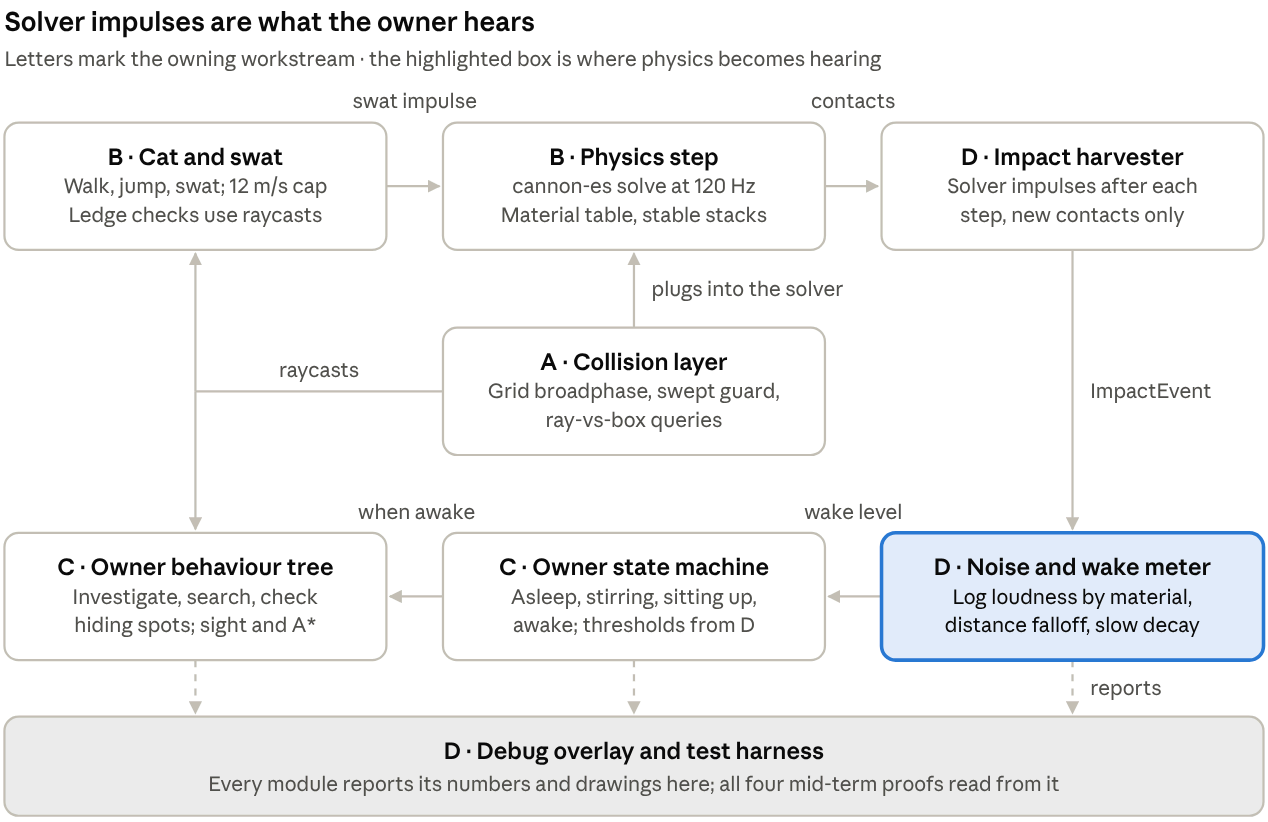
Each workstream maps to one pillar and one set of folders. The names are a proposal, so swap them at kickoff if strengths differ.

Workstream	Proposed owner	Pillar	W1–W2: playable by Oct 22	W3: complete by Nov 1	Review partner
A · Collision layer	Dharm	Collision Detection	Ray queries, grid broadphase, tunneling guard	Tunneling proof, fracture, hiding occluders, 400-body performance	B
B · Dynamics and cat	Antara	Physics & Dynamics	Physics world, material table, plate stack, cat, swat	Stacking proof, room props, cat polish	A
C · Owner AI	Christina	NPC AI / Behavior Trees	Owner state machine and poses, vision cone, stand-and-scan, caught rule	Behaviour tree, waypoint A*, hearing, hiding-spot checks	D
D · Core, noise and UI	Rushi	Couples physics to AI	Repo, game loop, overlay, impact harvester, noise, wake meter, HUD, score, round end, test harness	Coupling and performance proofs, lighting, integration	C

Review partners sit across each boundary: A and B share the solver, and C and D share the wake meter. Fracture moves from B to A in W3. A finishes its proof first, and fracture creates the 200–400 bodies that A's broadphase exists for.

Architecture

Simulation code never imports Three.js, so every proof runs headless in Node as well as in the browser.



module map · seven modules and the overlay

The top row is one physics step, and the right column carries each impact down to the owner. A's raycasts serve both the cat and the owner, and every box reports into D's overlay.

1. One loop owns time: a fixed 120 Hz step, two substeps per 60 Hz frame. Nobody else calls `world.step`.
2. Physics, collision, noise and AI are plain JavaScript plus cannon-es. Rendering reads their state and never writes it.
3. Modules talk through the contracts below and the event bus, never through each other's internals.
4. Every tuning number lives in `tuning.js` and shows in the overlay, so tuning never needs a code change.
5. Build in passes: stub, then real, then measured. Each pass merges on its own.

```
src/
  core/       loop, event bus, tuning.js, contracts      D
  physics/    world, materials, props                    B
  player/     cat controller, swat                       B
  collision/  grid broadphase, rays, guard, fracture     A
  noise/      impact harvester, loudness, wake meter     D
  ai/         owner state machine, behaviour tree, A*    C
  render/     scene, isometric camera, lights, sync      D
  ui/         HUD, debug overlay                         D
harness/      test harness page; scenarios by owners     D
tests/        headless Vitest suites                     all
```

Contracts between workstreams

These are the only places where one person's code touches another's. Each one starts as a stub on day one, so nobody waits for anyone.

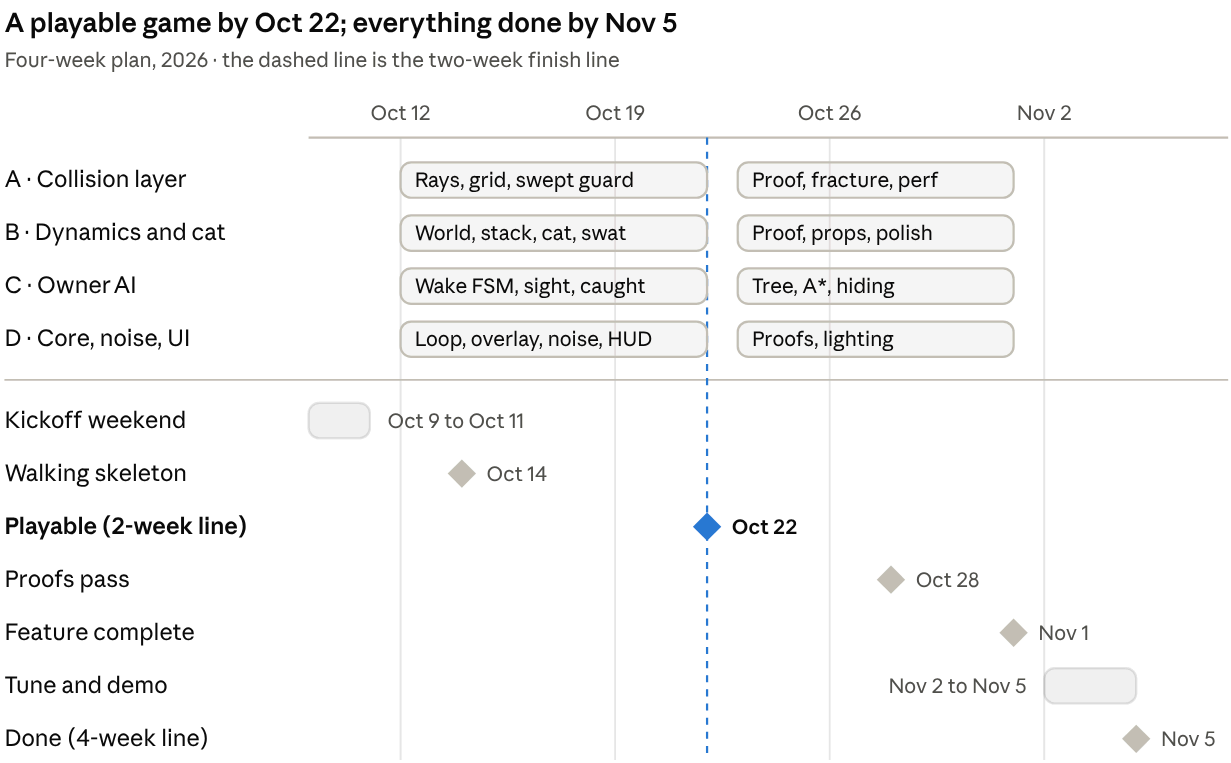
Contract	From → to	Shape	Day-one stub
`ImpactEvent`	D's harvester (reading B's world) → noise, score, fracture	`{bodyA, bodyB, impulse, point, normal, matA, matB}`, impulse in N·s	Overlay button that fires a fake impact
`NoiseEvent`	D's noise model → C's senses, overlay	`{position, loudness, heardLoudness}`	Fixed loudness per impact
Wake level	D's meter → C's state machine, HUD	`wake` from 0 to 1; stage thresholds live in `tuning.js`	Overlay slider
`raycast()`	A → B (ledge checks), C (sight), overlay	`raycast(origin, dir, maxDist, mask)` returns `{hit, distance, point, normal, body}`	Wraps cannon-es `world.raycastClosest`

Contract	From → to	Shape	Day-one stub
Static geometry	A → broadphase, guard, C's hiding checks	Room boxes, plus hiding-spot volumes	Hand-placed boxes
Material table	B → noise, fracture	Per material: friction, restitution, loudness, break impulse; per-pair overrides	Ceramic, wood, paper only
`swat()`	B → gameplay, test scenarios	`swat(target, direction, speed)`, exit speed capped at 12 m/s	Sets velocity directly
Owner state	C → HUD, round end, coupling test	`owner.state` plus a `stateChanged` event	Always "asleep"
`debug.register()`	D → everyone	Adds a number, a drawing layer or a tuning folder to the overlay	Ships by Oct 12
`Scenario`	Proof owners → D's harness	`{setup, step, metrics, pass}`; runs in the page and headless in Node	Ships by Oct 14

Shared conventions: units in m, kg and s; Y up; origin at the floor center; collision groups (`STATIC`, `PROP`, `DEBRIS`, `CAT`, `OWNER`) defined once in `core/`. Change a contract only after a heads-up in the group chat and approval from both sides.

Schedule

W1–W2 build a complete, playable round by Oct 22, W3 finishes the game and passes the proofs, and W4 is buffer until Nov 5.



plan timeline · four workstreams, kickoff to done

Every task in the workstream sections carries a week tag (W0–W4) that places it on this timeline. Bars that end at the dashed line must be done by the two-week line.

Workstream A — Collision layer (proposed: Dharm)

A answers every question about where things are: which bodies might touch, what a ray hits, and whether a fast body skipped through a surface. A's proof, due Oct 28, is the tunneling test.

- [ ] W0–W1 · Static geometry store: walls, floor, desk, bed frame and shelf kept apart from moving bodies, built once and never re-inserted per step.
- [ ] W1 · `raycast()` : ray-vs-AABB (slab test) and ray-vs-box (ray moved into the box's frame), unit-tested against cannon-es raycasts. B and C need it by Oct 14.
- [ ] W1 · Uniform-grid broadphase as a cannon-es `Broadphase` subclass, implementing both `collisionPairs()` and `aabbQuery()` , with cell size from `tuning.js` . Test: across 1,000 random scenes it never misses a pair the naive broadphase finds.
- [ ] W2 · Swept-sphere guard against static geometry, run after each substep. It only checks bodies moving more than 1 cm per substep (about 1.2 m/s, half the thinnest shelf), clamps a body to its first hit and removes the inward velocity.

- [ ] W2 · Overlay layers (occupied cells, candidate and narrow-phase pairs per step) and the tunneling scenario, written and running.
- [ ] W3 · Tunneling proof by Oct 28: 100 swats at 12 m/s into a 2 cm shelf with zero pass-throughs, then raise the speed until it fails and log that speed. Also run B's stack with the guard on and off; the numbers must match.
- [ ] W3 · Fracture for mugs and plates: 2–4 pre-cut pieces each, swapped in when an impact passes the break impulse, inheriting linear and angular velocity.
- [ ] W3 · Hiding-spot occluders (bed skirt, desk) and the hiding volumes C checks.
- [ ] W3–W4 · 200–400 bodies: tune cell size, keep narrow-phase pairs under about 200 per step, profile in Chrome and Firefox.

Watch out: cannon-es's own `world.raycast*` methods call the broadphase's `aabbQuery()`. The base class only logs a warning and returns nothing, so without it every built-in raycast silently misses once the grid is plugged in.

Workstream B — Dynamics and cat (proposed: Antara)

B owns how things move: the physics world, what everything is made of, and the cat. B's proof, due Oct 28, is the stacking test.

- [ ] W0–W1 · Physics world wrapper: gravity, solver iterations and contact stiffness read from `tuning.js`, with one `step()` that only D's loop calls.
- [ ] W1 · Material table v1 (ceramic, wood, paper, plastic, fabric, cat) with friction and restitution per pair, plus the loudness and break-impulse columns that D and A read.
- [ ] W1 · Prop factories for plate (low-sided convex cylinder), mug and book, each visual mesh built from its collider's dimensions. Slice scene: five plates, two mugs and a book on A's desk, with C's bed and owner across the room.
- [ ] W1 · Cat controller: dynamic body with fixed rotation, WASD turned to match the isometric camera, ground check by ray, jump. Needed for the Oct 14 skeleton.
- [ ] W1 · Swat: a paw volume in front of the cat applies an impulse capped so the target leaves at 12 m/s or less. Test scenarios call it too.
- [ ] W1–W2 · Stack tuning with sleeping off: solver iterations, contact stiffness and relaxation, friction, plate side count, all live in the overlay. Decide plates or books by Oct 21.
- [ ] W2 · Stacking scenario written and running: 60 s, max penetration under 2 mm, drift under 5 mm.
- [ ] W3 · Stacking proof passing by Oct 28.

- [ ] W3 · Room clusters: bookshelf with books, nightstand with lamp, plant pot, each with a material.
- [ ] W3 · Cat polish: a drop shadow, and a landing marker placed by a ledge ray.
- [ ] W4 · Stability pass with the full room and fracture debris.

Watch out: thin cylinders are the hardest shape to stack in cannon-es. More sides mean more contact points per pair and slower solver convergence, so keep the collider coarse (8–12 sides) and the visual mesh round.

Workstream C — Owner AI (proposed: Christina)

C owns what the owner perceives and decides. The Oct 22 game only needs the owner to wake, stand and scan, so C builds the behaviour tree and waypoint A* headless in W2 and wires them in during W3.

- [ ] W0–W1 · Owner state machine: asleep, stirring, sitting up, awake, driven by D's wake level and thresholds. A minimum time in each stage prevents flicker; unit-tested in Node.
- [ ] W1 · Owner model on the bed: low-poly segmented figure with no rigging, one pose per stage, blended transitions.
- [ ] W1–W2 · Vision cone: angle, range and eye height in `tuning.js`. Sight is confirmed by A's rays to three points on the cat, and the cone and rays draw in the overlay.
- [ ] W2 · Stand and scan: once awake, the owner stands by the bed and sweeps the cone across the room. With the caught rule, this makes the Oct 22 game playable without the tree.
- [ ] W2 · Caught rule: the awake owner keeps the cat in sight for a set time.
- [ ] W2 · Behaviour-tree runtime (selector, sequence, inverter, condition and action nodes, a blackboard, a RUNNING status, ticked at 10 Hz) and A* over hand-placed waypoints, both unit-tested in Node.
- [ ] W2–W3 · Tune stage thresholds with D for the coupling proof.
- [ ] W3 · Owner tree: investigate the last noise, search nearby, check hiding spots, return to bed.
- [ ] W3 · Walking: a kinematic body follows the waypoint path with a simple walk cycle and pushes debris aside.
- [ ] W3 · Hearing radius once awake, and the hiding-spot check: the owner crouches to a lower eye height and re-tests sight, so hiding works through geometry rather than a flag.

- [ ] W4 · Tuning from playtests, and the tree's active path shown in the overlay.

Navigation isn't in A2, but the owner can't investigate a noise across the room without it. That's why waypoint A* is built in W2, before the tree needs it.

Workstream D — Core, noise, UI and harness (Rushi)

D owns the frame every other module runs inside, plus the noise path that couples physics to the owner. The debug overlay ships first because all four proofs read from it.

- [ ] W0 · Repo: Vite, Three.js, cannon-es, Vitest, lil-gui and stats.js; folder layout; event bus; `tuning.js`; every contract stubbed; Netlify deploy previews; CI running the headless tests.
- [ ] W0–W1 · Game loop: fixed 120 Hz step with an accumulator, a cap on steps per frame, and render interpolation. Hooks after each step run the guard, harvester and AI.
- [ ] W1 · Renderer scaffold (scene, fixed isometric orthographic camera, body-to-mesh sync) and debug overlay v1 (FPS, stat panels, tuning folders, contact points and normals, bounding boxes, `debug.register()`).
- [ ] W1 · Walking skeleton by Oct 14: a swatted stub mug makes noise and changes the owner's pose, live in the deployed build.
- [ ] W1 · Impact harvester: after each step, read each new contact's solver impulse (`multiplier × dt`) from `world.contacts`, sum it per pair and emit `ImpactEvent`. B reviews.
- [ ] W1–W2 · Noise model and wake meter: loudness grows with the log of impulse and is scaled per material pair. It falls off with distance to the bed, impacts under a minimum make no noise, and the meter decays slowly.
- [ ] W2 · HUD (wake meter and chaos score), chaos scoring, and round end: caught or the 3-minute timer, then restart. The game is playable by Oct 22.
- [ ] W2 · Test harness page: lists scenarios, runs them, shows live overlay numbers and pass or fail, and exports a log.
- [ ] W2–W3 · Coupling scenario with C (the same swat reaches sitting up beside the bed and stays at or below stirring from the far wall) and the silent-stack check (B's resting stack makes zero noise for 60 s).
- [ ] W3 · Coupling and performance proofs passing on Chrome and Firefox by Oct 28.
- [ ] W3 · Lighting: one shadow-casting lamp as the warm key, a cool moonlight fill, a faked light shaft through the window, and wood and fabric textures.

- [ ] W4 · Integration, performance pass, demo recording and report by Nov 5. Could-haves only if everything else is done.

Watch out: don't take impulses from cannon-es's `collide` event. It fires before the solver runs, so the impulse isn't computed yet. It also fires on both bodies, so a naive listener counts every crash at least twice.

Mid-term proofs

Each proof has one owner, who writes its scenario for D's harness. All four must pass on Chrome and Firefox by Oct 28, which leaves W4 free for tuning.

Proof	Owner	Support	Pass criterion	Measured in
Stacking	B	D (overlay)	Five plates, sleeping off, 60 s: max penetration under 2 mm, stack drift under 5 mm	Overlay and harness log
Tunneling	A	B (`swat()`)	100 swats at 12 m/s into a 2 cm shelf, zero pass-throughs; the failure speed is found and reported	Harness log, headless
Coupling	D	C (state machine)	The same swat reaches "sitting up" from beside the bed and goes no further than "stirring" from the far wall	Harness log and a recording
Performance	D	Everyone	60 FPS with 60 awake bodies while the other three run	Overlay FPS

The tunneling test is a real test: at 120 Hz a 12 m/s mug moves 10 cm per step, five times the shelf's thickness, so discrete collision alone can miss it.

Working agreements

These are the rules that keep four people merging into one codebase without breaking each other's work.

- **Repo:** GitHub, with `main` always runnable. Netlify deploys `main` and a preview for every pull request.

- **Branches:** one per task, prefixed by workstream, such as `a/grid-broadphase` or `c/bt-runtime`.
- **Reviews:** one approval from your review partner (A with B, C with D). Contract changes need all four.
- **CI:** the headless tests run in Node with Vitest and must pass before a merge. Each proof becomes a test as soon as it exists.
- **Cadence:** a written check-in every evening (done, next, blocked) and two 20-minute calls a week, Monday and Thursday. Merge to main at least daily, and play the latest build together on every Thursday call.
- **Definition of done:** merged; covered by a unit test or a number in the overlay; constants in `tuning.js`; no console errors in Chrome or Firefox.
- **AI-written code:** allowed, as A2 says. Anything that touches collision or noise is reviewed and tested by that module's owner.

Risks and fallbacks

The biggest risks are the pace and the plate stack, so both get early tripwires. Every risk below has a fallback that protects the Oct 22 game.

Risk	Early sign	Fallback	Owner
The pace slips: four weeks nearly doubles everyone's hours	A missed evening check-in, or a task stuck two days running	The review partner takes over the stub, and the team cuts the next item it can live without	Rushi
Plates won't settle	Drift over 5 mm by Oct 19	Switch the stack to books by Oct 21 and keep plates as a stretch	B
Integration lands late	Walking skeleton not running by Oct 14	Everyone pauses feature work until it runs	D
Guard jitters resting stacks	Stack numbers change with the guard on	Raise the speed gate; give thin shelves thicker colliders than their meshes	A

Risk	Early sign	Fallback	Owner
Noise double-counts or reads zero impulse	Meter jumps twice per crash	Harvest from `world.contacts` after the step, never in the `collide` event	D
Custom grid looks slower than built-in at 60 bodies	Step time in the overlay	Expected at that count; judge it at 200–400 bodies and keep the naive broadphase as a switchable reference	A
Behaviour tree slips	Owner not walking the waypoints by Oct 30	Stand-and-scan from W2 stays as the awake behaviour, and caught still works	C
Shadows plus 400 bodies drop below 60 FPS	Frame time in W3	One shadow-casting light (the lamp), moonlight without shadows, instanced debris	D
Jumps are hard to judge in isometric view	Playtesters miss jumps	Drop shadow and landing marker under the cat	B

Decisions to settle at kickoff

Eight calls to make at kickoff this weekend; everything else in this plan can start without them.

- [ ] Commit to the four-week target: playable by Oct 22, done by Nov 5.
- [ ] Check that everyone can carry nearly double the weekly hours for four weeks; if not, say so now and cut more.
- [ ] Confirm the course mid-term date (assumed Nov 6) and any later checkpoints the remaining weeks should cover.
- [ ] Confirm who takes workstreams A, B and C (proposal above).
- [ ] Decide whether the owner can see while sitting up, or only once standing.
- [ ] Set the caught rule: how many seconds in sight counts as caught.
- [ ] Fix the controls: WASD to move, Space to jump, J or click to swat.

- [ ] Clean up A2 before submitting: fill in the ownership names and the mentor (Danrui Li), and say that cannon-es does the narrow phase.